

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет по лабораторной работе №3

«Разработка одностраничного веб-приложения (SPA) с использованием
Vue.js»

Выполнил:

Конькова Нина

Группа J3210

Проверил:

Добряков Д. И.

Содержание

1	Задача	2
2	Используемые технологии	2
3	Структура проекта	2
4	Ход работы	2
4.1	Настройка SPA и маршрутизации (Vue Router)	2
4.2	Взаимодействие с внешним API через axios	3
4.3	Компонентный подход и переиспользование	4
4.4	Composables: вынесение повторяющейся логики	5
4.5	Дополнительная функциональность (Chart.js и уведомления)	5
5	Запуск проекта	6
6	Вывод	6

1 Задача

Цель лабораторной работы — миграция ранее разработанного приложения (ЛР1–ЛР2) на фреймворк **Vue.js** и реализация архитектуры **SPA**. Приложение должно удовлетворять требованиям:

1. Подключение и настройка **Vue Router** для навигации без перезагрузки страницы.
2. Реализация работы с **внешним API** (желательно через **axios**).
3. **Компонентный подход**: разумное разделение UI на переиспользуемые компоненты и страницы (views).
4. Использование **composables** для вынесения повторяющейся логики в отдельные модули.

2 Используемые технологии

- Vue.js 3 (подключение через CDN).
- Vue Router 4 (маршрутизация SPA).
- Axios (HTTP-запросы).
- Bootstrap 5 и Bootstrap Icons (верстка и иконки).
- Chart.js (визуализация метрик на странице эксперимента).

3 Структура проекта

Код приложения разделен на следующие части:

- `src/main.js` — точка входа, инициализация приложения и роутера.
- `src/views/*` — страницы (Dashboard, Experiments, Models, Artifacts, Login, Register и др.).
- `src/components/*` — переиспользуемые компоненты (Layout, Sidebar, TopBar, карточки и т.д.).
- `src/composables/*` — переиспользуемая логика (auth, theme, toasts, filters, API).
- `json/db.json` — локальные данные пользователей (используются при авторизации).

4 Ход работы

4.1 Настройка SPA и маршрутизации (Vue Router)

В `src/main.js` создан роутер на основе `createWebHashHistory()`, что упрощает запуск без сложной серверной конфигурации. Для страниц добавлены метаданные **meta**:

- `requiresAuth` — доступ только для авторизованных пользователей.
- `guestOnly` — доступ только для гостей (страницы входа/регистрации).

- `title` — заголовок вкладки браузера.

Маршруты защищены глобальным `guard`-ом `router.beforeEach()`, который при отсутствии авторизации выполняет редирект на `/login` и сохраняет целевой маршрут в `query.redirect`.

Листинг 1: Фрагмент настройки роутера и `guard`-логики (`src/main.js`)

```
const routes = [
  { path: '/login', component: LoginView, meta: { guestOnly: true } },
  { path: '/dashboard', component: DashboardView, meta: { requiresAuth: true } },
  { path: '/experiments/:id', component: ExperimentDetailView, meta: { requiresAuth: true } }
];

router.beforeEach((to) => {
  const { isAuthenticated } = useAuth();

  if (to.meta.requiresAuth && !isAuthenticated.value) {
    return { path: '/login', query: { redirect: to.fullPath } };
  }

  if (to.meta.guestOnly && isAuthenticated.value) {
    return '/dashboard';
  }

  return true;
});
```

4.2 Взаимодействие с внешним API через `axios`

Для выполнения требования работы с внешним API был выбран **Hugging Face API**. Реализован `composable useHuggingFaceModels`, который:

- хранит состояние `models`, `isLoading`, `error`;
- выполняет GET-запрос через `axios`;
- преобразует ответ API в удобный для интерфейса формат.

Данные отображаются на Dashboard в компоненте `HuggingFaceModels` в виде сетки карточек (`featured + secondary + compact`).

Листинг 2: Запрос к Hugging Face API (`useHuggingFaceModels.js`)

```
const api = window.axios;
const HUGGING_FACE_URL =
  'https://huggingface.co/api/models?sort=downloads&direction=-1&limit=8';

async function fetchModels() {
  const response = await api.get(HUGGING_FACE_URL);
  models.value = response.data.map((model) => ({
    id: model.id || model.modelId,
    name: (model.modelId || '').split('/').pop(),
```

```

    downloads: model.downloads || 0
  }));
}

```

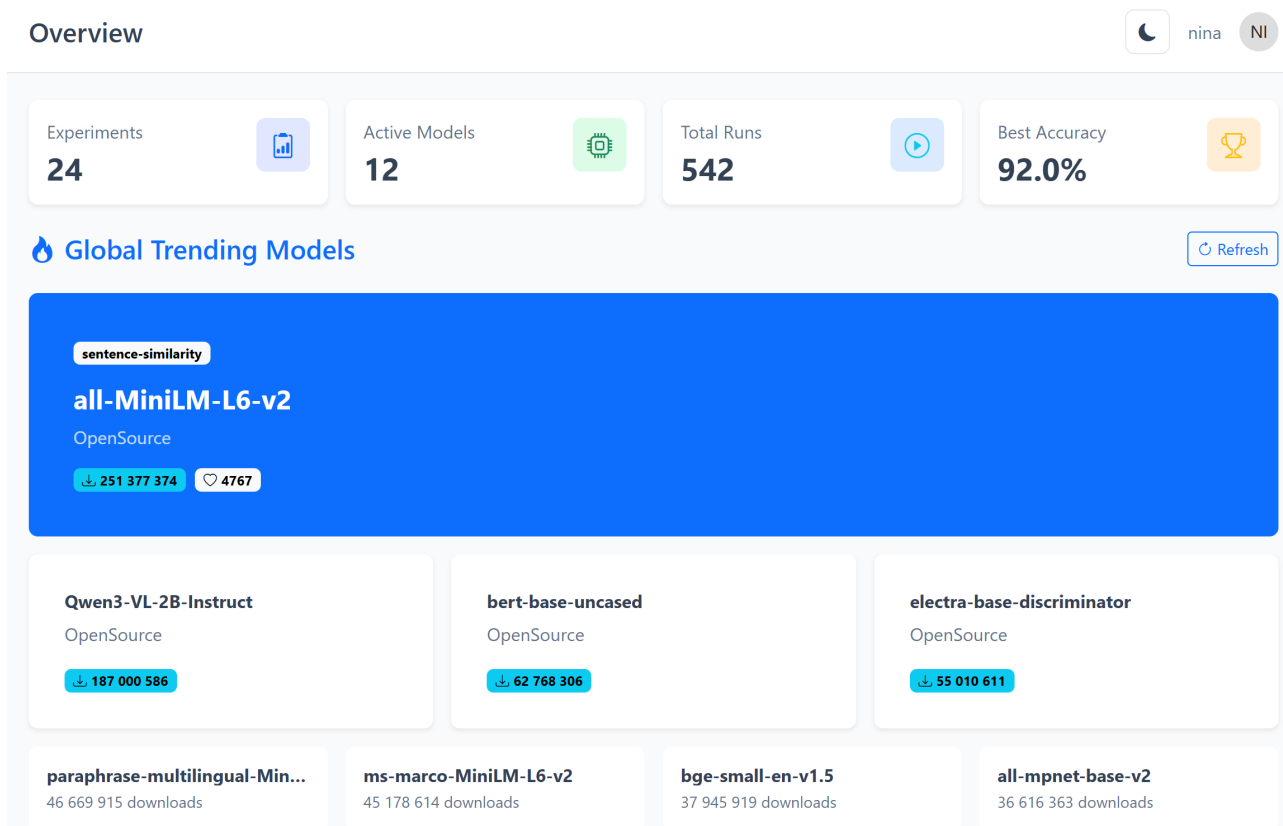


Рис. 1: Dashboard: блок “Global Trending Models” с данными Hugging Face API

4.3 Компонентный подход и переиспользование

В ходе миграции интерфейс приложения был разбит на компоненты:

- **AppLayout** — каркас приложения (Sidebar + TopBar + слот контента).
- **SidebarNav** — боковое меню навигации.
- **TopBar** — верхняя панель (заголовок страницы, тема, данные пользователя).
- **StatCard** — переиспользуемая карточка статистики.
- **ToastHost** — контейнер уведомлений (toast-сообщений).
- **HuggingFaceModels** — компонент, отображающий результаты внешнего API.

Пример: **AppLayout** использует **slot**, чтобы все страницы имели единый макет, но разное содержимое.

Листинг 3: Компонент каркаса приложения (components/AppLayout.js)

```

template: `
  <div class="d-flex app-shell" id="wrapper">
    <SidebarNav />

```

```

<div id="page-content-wrapper" class="w-100">
  <TopBar :title="title" :user="currentUser" />
  <main class="container-fluid_p-4">
    <slot></slot>
  </main>
</div>
</div>

```

4.4 Composables: вынесение повторяющейся логики

Для соблюдения требований лабораторной работы повторяющиеся части логики вынесены в composables:

1. `useAuth` — авторизация, регистрация, выход, хранение состояния в `localStorage`. Для загрузки базового набора пользователей применяется запрос к локальному JSON-файлу `json/db.json` через `axios`.
2. `useTheme` — переключение темы и сохранение выбора в `localStorage`.
3. `useToast` — добавление/удаление уведомлений с авто-закрытием.
4. `useExperimentFilters` — фильтрация списка экспериментов через `computed` (поиск, статус, дата, точность).
5. `useHuggingFaceModels` — внешний API (Hugging Face).

Листинг 4: Пример фильтрации через `computed` (`useExperimentFilters.js`)

```

const filteredExperiments = computed(() => {
  return experiments.filter((experiment) => {
    const matchesSearch = experiment.name
      .toLowerCase().includes(search.value.trim().toLowerCase());
    const matchesStatus = status.value === 'All' || experiment.status
      === status.value;
    const matchesDate = !date.value || experiment.date === date.value;
    const matchesAccuracy = experiment.accuracy >= Number(minAccuracy.
      value);
    return matchesSearch && matchesStatus && matchesDate &&
      matchesAccuracy;
  });
});

```

4.5 Дополнительная функциональность (Chart.js и уведомления)

На странице деталей эксперимента реализованы:

- график метрики обучения через `Chart.js`;
- модальное окно логов;
- уведомления (`toast`) о действиях пользователя (например, “Downloading model.pth”).

5 Запуск проекта

Проект запускается локальным Node.js сервером, который раздает статические файлы и корректно обрабатывает маршруты SPA:

- Команда: `npm run dev`
- Сервер: `server.mjs` (fallback на `index.html` для неизвестных путей).

6 Вывод

В рамках лабораторной работы №3 выполнена миграция проекта на Vue.js и реализована архитектура SPA. Настроена маршрутизация с защитой страниц через Vue Router guards, выполнена интеграция с внешним API (Hugging Face) посредством axios, интерфейс разделен на переиспользуемые компоненты, а повторяющаяся логика вынесена в composables (auth, theme, toasts, filters, API). Полученное приложение готово к дальнейшему расширению (подключение реального backend, расширение сущностей экспериментов и моделей).